Object Oriented Programming :: Encapsulation

Ajith de Silva @epita.fr | 2026

OOP

The Four Pillars
OOP
Abstraction
Inheritance
Polymorphism
Encapsulation

OOP :: Class/Object/Instance

What is Class ???

- A Template or Blueprint
- Contains collection of properties & methods
- Helps to build Objects with different characteristics & behaviors

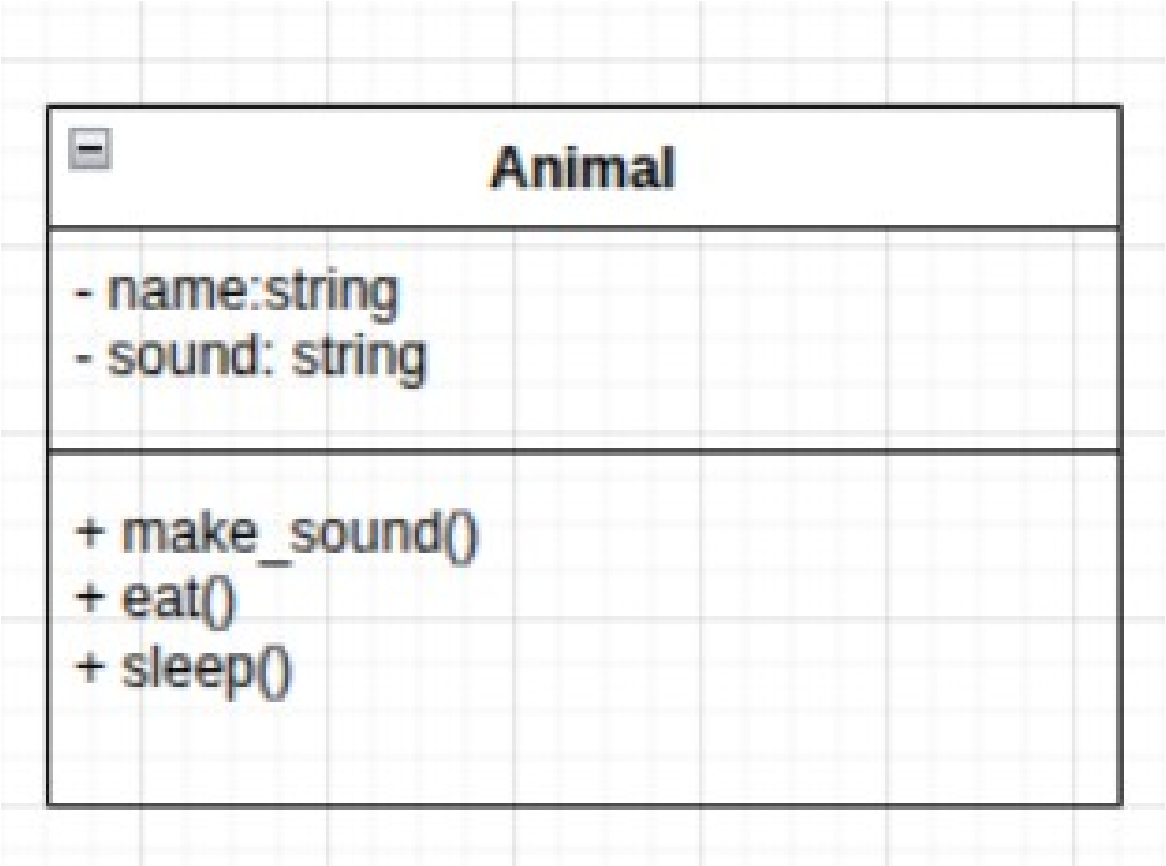
What is Object ??

- Entity that we implement in SW
- Also called "**Object of Class**"
- Used to create Instances

What is Instance ???

- Virtual copy of the Object
- A given/achieved object is **Instance**
- When create **Instance** of class, distinct **Objects** will be created in Memory.

Object Oriented Programming :: Class



HAS
TYPE - > Animal

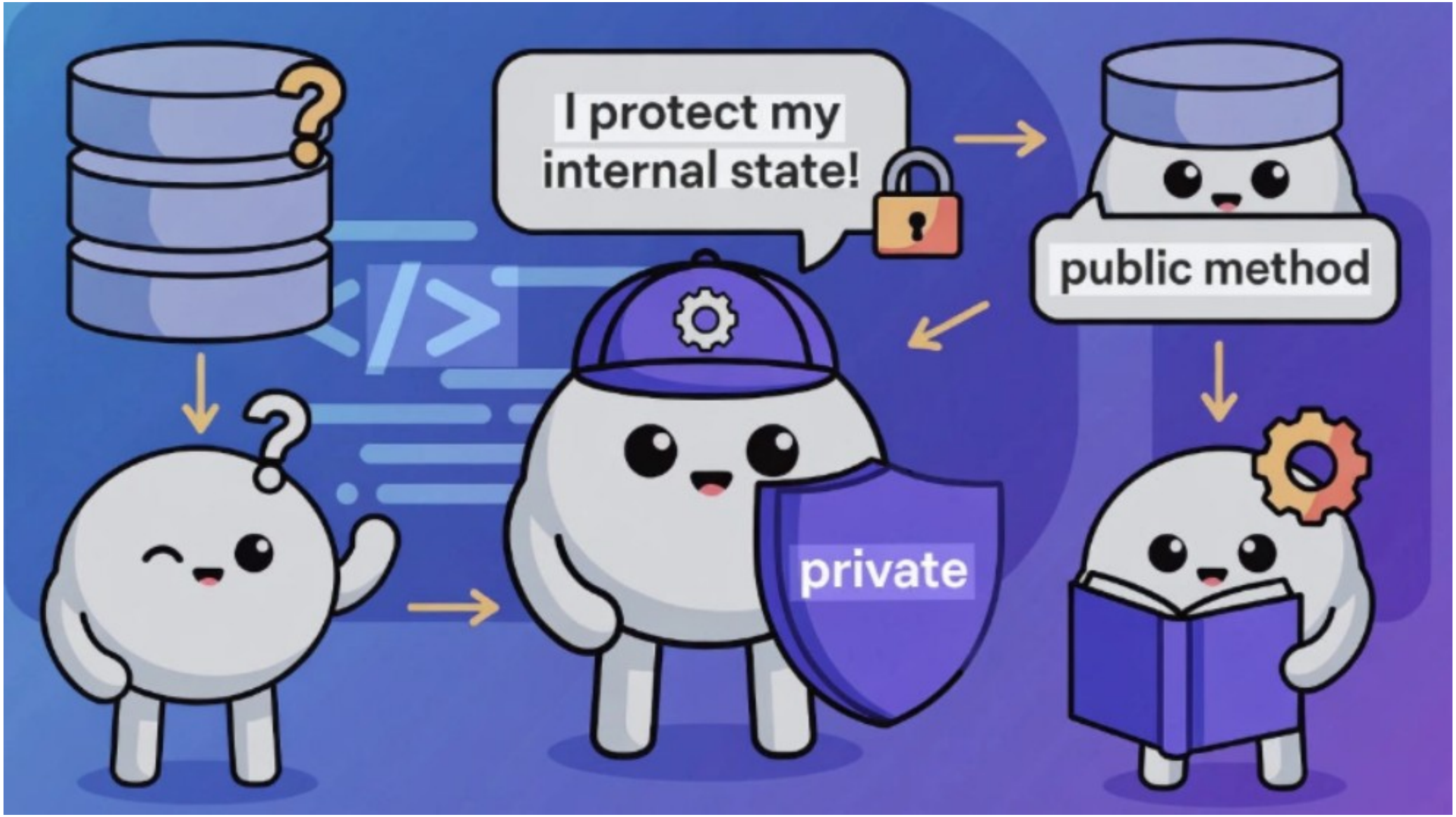
HAS Private variables " - "
HAS Public variables " + "

HAS Private methods " - "
HAS Public methods " + "

** Variable ??
Can hold some value

** Property ??
An attribute of a variable

Object Oriented Programming :: Encapsulation



Object Oriented Programming :: Encapsulation

*** Data Hiding ***

SIMPLE ::

Will include

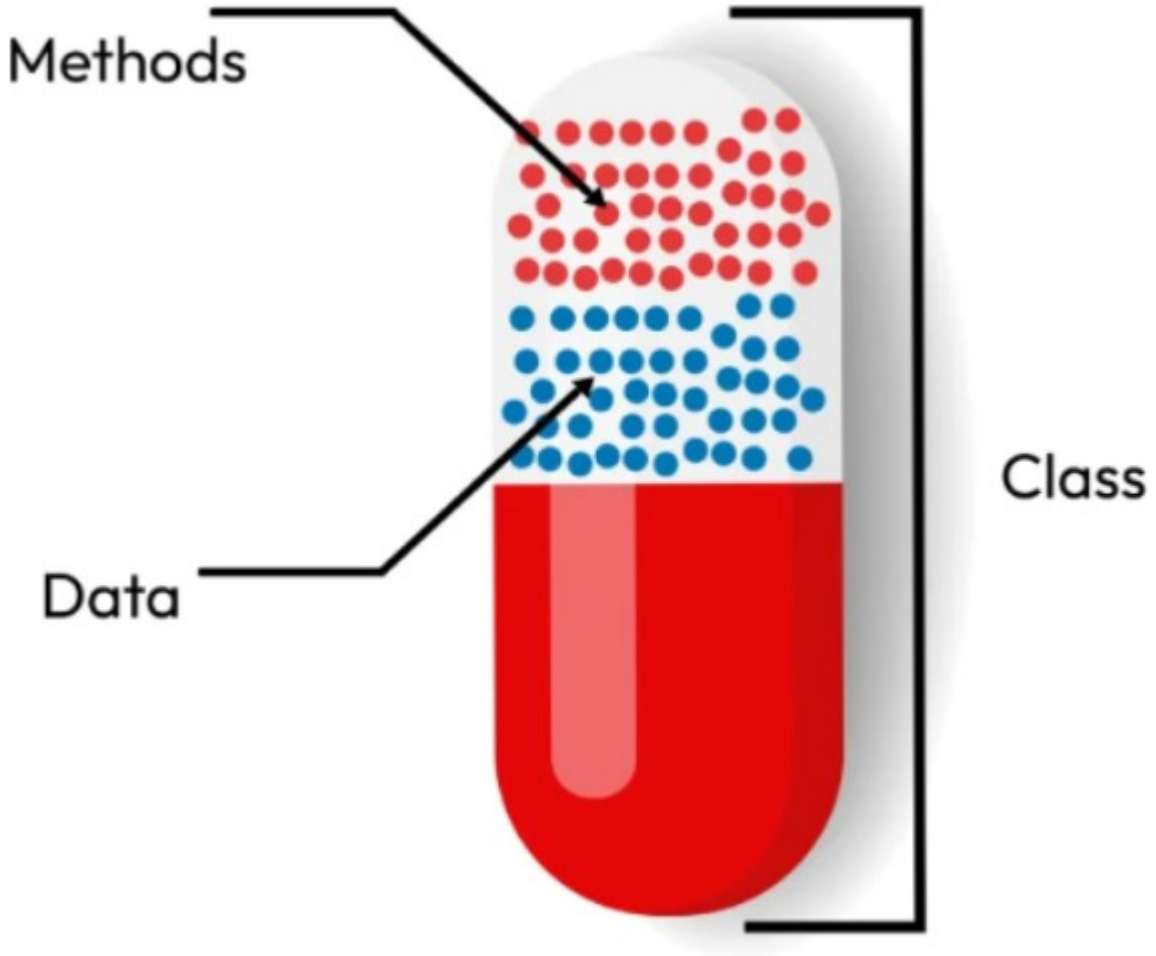
Data

Methods

Inside the class and let class methods to use

Data what we handed over.

*** So it is using the methods in-side of the clas



Object Oriented Programming :: Encapsulation

Private Variables:

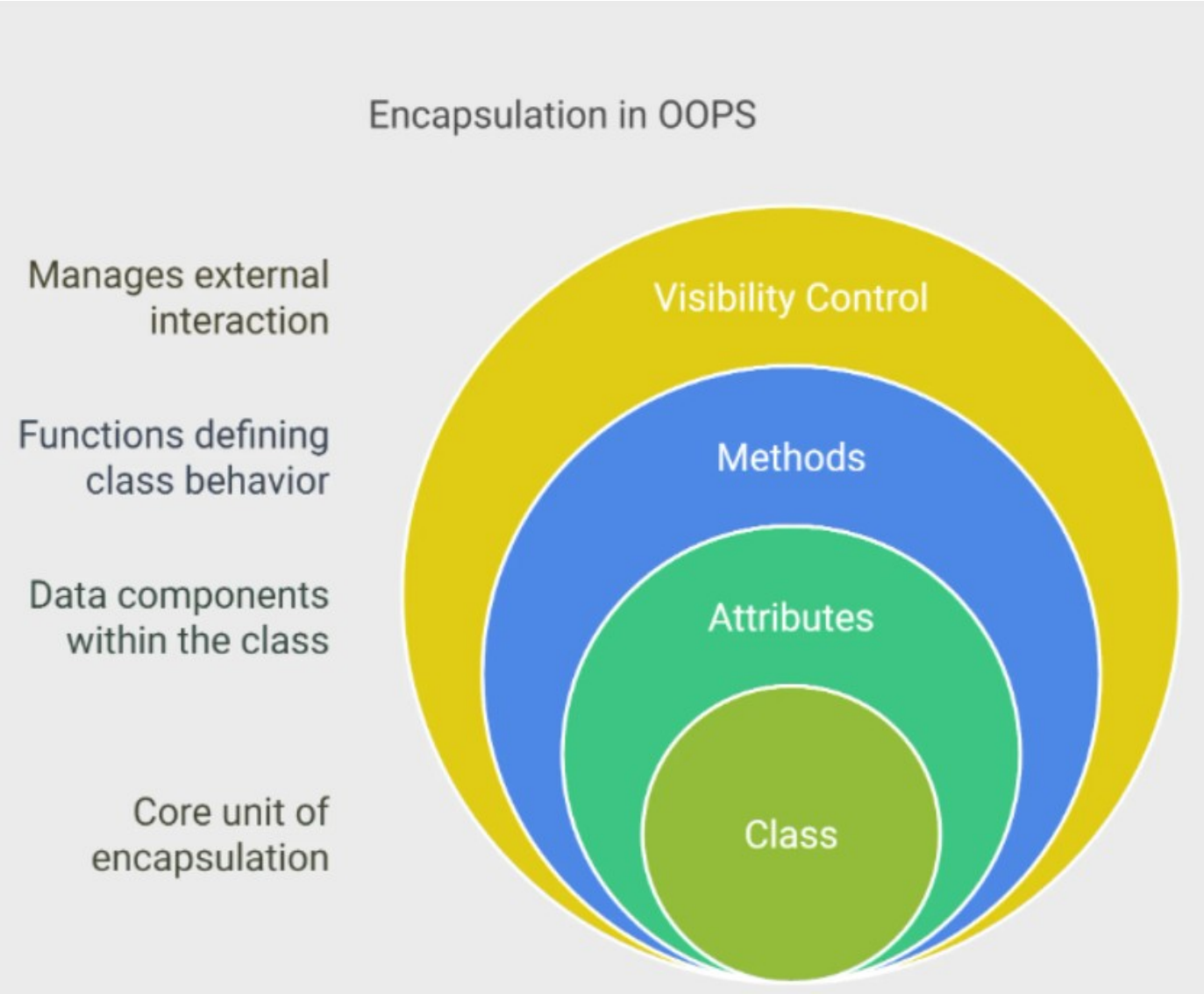
keep information hidden. No one access them directly

Public Methods:

create Getters/Setters to handler private variables. Control before action.

Data Consistency:

all the data the code needs for its process is inside the class with control.



Object Oriented Programming :: Encapsulation

Using –
private variable
getter method

```
BankAccount
- __balance : float
+ name : str
+ __init__(name, balance)
+ deposit(amount)
+ withdraw(amount)
+ get_balance() : float
```

clear
less Pythonic**

Using –
private variable
getter/setter property

```
BankAccount
- __balance : float
+ name : str
+ __init__(name, balance)
+ balance() : float
+ balance(amount)
+ deposit(amount)
+ withdraw(amount)
```

looks like variable
more Pythonic

Object Oriented Programming :: Encapsulation

Using –
private variable
getter method

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    def set_balance(self, amount):
        if amount >= 0:
            self.__balance = amount
```

Using –
private variable
getter/setter property

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, amount):
        if amount >= 0:
            self.__balance = amount
```